FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Nemeno, Faith S.
Section:	AN43
Professor:	Ms. Crisola G. Tan
Date:	October 5, 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

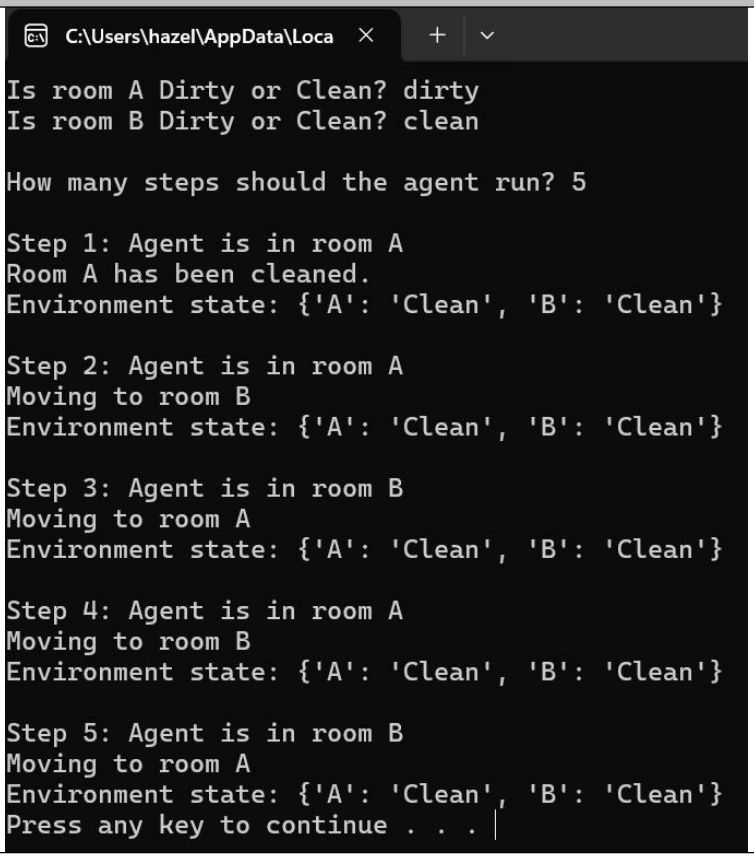
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

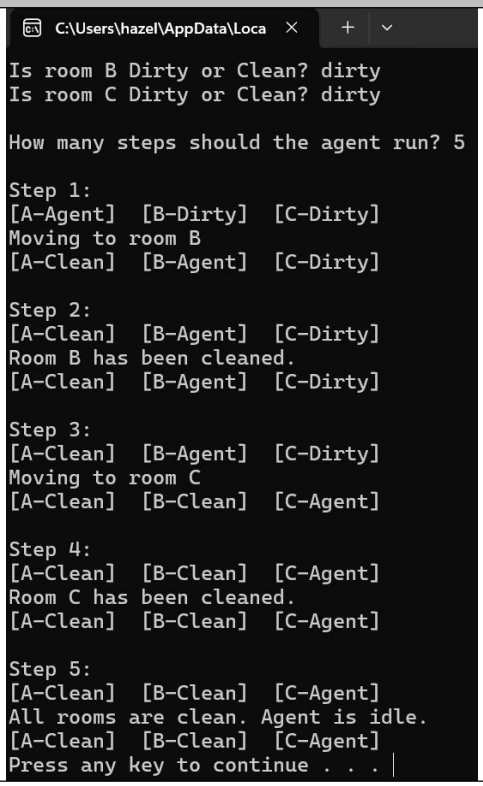
1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

Source Code - Two Rooms:

```
class Environment:
    """Class representing the two-room environment."""
    def __init__(self, room_a_state: str, room_b_state: str):
        # Store state in a dictionary as required by Task 1
        self.rooms = {
            'A': room_a_state.capitalize(),
            'B': room_b_state.capitalize()
        }
```

```
    def is_dirty(self, room: str) -> bool:
        """Check if a given room is dirty."""
        return self.rooms.get(room) == "Dirty"

    def clean_room(self, room: str) -> None:
        """Clean a given room."""
        self.rooms[room] = "Clean"


class RuleBasedAgent:
    """Class representing the Rule-Based Vacuum Cleaner Agent."""
    def __init__(self, start_location: str):
        self.current_location = start_location

    def perceive_and_act(self, env: Environment) -> str:
        """Perceive current state and decide on an action based on rules."""
        # Rule 1: If current room is dirty, clean it
        if env.is_dirty(self.current_location):
            env.clean_room(self.current_location)
            return f"{self.current_location} has been cleaned"

        # Rule 2: If current room is clean, move to the other room
        else:
            self.move()
            return f"Moving to Room {self.current_location}"

    def move(self) -> None:
        """Switch agent location between Room A and Room B."""
        if self.current_location == 'A':
            self.current_location = 'B'
        else:
            self.current_location = 'A'


def main():
    print("=== Rule-Based Vacuum Cleaner Simulation ===")

    # Task 1: Initialize Environment from User Input
    state_a = input("Enter status for Room A (Clean/Dirty): ").strip()
    state_b = input("Enter status for Room B (Clean/Dirty): ").strip()
    start_loc = input("Enter starting room for Agent (A/B): ").strip().upper()

    env = Environment(state_a, state_b)
    agent = RuleBasedAgent(start_loc)

    # Task 3: Input step count and run simulation
    steps = int(input("\nHow many steps should the agent run: "))

    for step in range(1, steps + 1):
```

```
        print(f"\nStep {step}:")
        print(f"  Agent Location : Room {agent.current_location}")

        # Execute perceive-and-act cycle
        action = agent.perceive_and_act(env)

        print(f"  Room {action}")
        print(f"  Environment State: {env.rooms}")


if __name__ == "__main__":
    main()
```

Screenshot:

```
=== Rule-Based Vacuum Cleaner Simulation ===
Enter status for Room A (Clean/Dirty): dirty
Enter status for Room B (Clean/Dirty): clean
Enter starting room for Agent (A/B): A

How many steps should the agent run: 5

Step 1:
  Agent Location : Room A
  Room A has been cleaned
  Environment State: {'A': 'Clean', 'B': 'Clean'}

Step 2:
  Agent Location : Room A
  Room Moving to Room B
  Environment State: {'A': 'Clean', 'B': 'Clean'}

Step 3:
  Agent Location : Room B
  Room Moving to Room A
  Environment State: {'A': 'Clean', 'B': 'Clean'}

Step 4:
  Agent Location : Room A
  Room Moving to Room B
  Environment State: {'A': 'Clean', 'B': 'Clean'}

Step 5:
  Agent Location : Room B
  Room Moving to Room A
  Environment State: {'A': 'Clean', 'B': 'Clean'}
```

Source Code - Three Rooms:

```
class Environment:
    """Class representing the three-room environment."""
    def __init__(self, room_a: str, room_b: str, room_c: str):
        self.rooms = {
            'A': room_a.capitalize(),
            'B': room_b.capitalize(),
            'C': room_c.capitalize()
        }

    def is_dirty(self, room: str) -> bool:
        """Check if a room is dirty."""
        return self.rooms.get(room) == "Dirty"

    def clean_room(self, room: str) -> None:
        """Clean a room."""
        self.rooms[room] = "Clean"

    def all_clean(self) -> bool:
        """Check if all rooms in the environment are clean."""
        return all(status == "Clean" for status in self.rooms.values())


class RuleBasedAgent:
    """Class representing the Rule-Based Vacuum Cleaner Agent."""
    def __init__(self, start_location: str = 'A'):
        self.current_location = start_location

    def perceive_and_act(self, env: Environment) -> str:
        """Decide action based on rules."""
        # Check if all rooms are clean
        if env.all_clean():
            return "All rooms are clean. Agent is idle."

        # Rule 1: If current room is dirty, clean it
        if env.is_dirty(self.current_location):
            env.clean_room(self.current_location)
            return f"Room {self.current_location} has been cleaned."

        # Rule 2: If current room is clean, move to the next room
        else:
            self.move()
            return f"Moving to room {self.current_location}"

    def move(self) -> None:
        """Cycle agent through rooms A -> B -> C -> A."""
        if self.current_location == 'A':
            self.current_location = 'B'
        elif self.current_location == 'B':
            self.current_location = 'C'
```

```
        else:
            self.current_location = 'A'


def display_grid(env: Environment, agent: RuleBasedAgent) -> str:
    """For [A-Clean] [B-Agent] [C-Dirty]."""
    grid_items = []
    for room in ['A', 'B', 'C']:
        if agent.current_location == room:
            grid_items.append(f"[{room}-Agent]")
        else:
            grid_items.append(f"[{room}-{env.rooms[room]}]")
    return " ".join(grid_items)


def main():
    # for room statuses
    state_a = "Clean"
    state_b = input("Is room B Dirty or Clean? ").strip()
    state_c = input("Is room C Dirty or Clean? ").strip()

    env = Environment(state_a, state_b, state_c)
    agent = RuleBasedAgent(start_location='A')

    steps = int(input("\nHow many steps should the agent run? "))

    for step in range(1, steps + 1):
        print(f"\nStep {step}:")

        # Display state BEFORE action
        print(display_grid(env, agent))

        # Agent perceives environment and decides action
        action_msg = agent.perceive_and_act(env)
        print(action_msg)

        # Display state AFTER action
        print(display_grid(env, agent))


if __name__ == "__main__":
    main()
```

Screenshots:

```
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty

How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Dirty] [C-Dirty]
Moving to room B
[A-Clean] [B-Agent] [C-Dirty]

Step 2:
[A-Clean] [B-Agent] [C-Dirty]
Room B has been cleaned.
[A-Clean] [B-Agent] [C-Dirty]

Step 3:
[A-Clean] [B-Agent] [C-Dirty]
Moving to room C
[A-Clean] [B-Clean] [C-Agent]

Step 4:
[A-Clean] [B-Clean] [C-Agent]
Room C has been cleaned.
[A-Clean] [B-Clean] [C-Agent]

Step 5:
[A-Clean] [B-Clean] [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?

   For rule 1, the agent should clean the room right away, and for rule 2, if the room it is currently in is already clean then the agent should move to another room.

2. How does the agent decide when to clean and when to move?

   Every time a stepp starts, the vacuum checks the status of its current room. When dirt is detected, it should clean. It only moves when the room is already clean.

3. What happens if both rooms are already clean at the start?

   If both rooms are already clean, then the agent will move in a loop between the rooms as indicated by the amount of steps given.

4. How would you modify the agent to handle **more than two rooms**?

   Putting more rooms in the code and adding a condition to the if-else statement wherein the agent will be able to move between more rooms rather than just two.

Answer the Questions ⇩